Parallelization in `loo` (current state)

Stan GSOC 2026

What is parallelization?

We open with a broad summary of what parallel computing is, why we want to parallelize, and some drawbacks/considerations. This section can be skipped if one is already familiar with these matters; more in-depth treatments of these topics can be found in the main references for this section [1], [2], [3].

Parallel programs are programs which are written to exploit hardware level parallelism [3]. Spelled out, we are trying to take some chunk of work which would otherwise run one step at a time, split it into pieces, and run some of those pieces at the same time. This is useful when the work is large enough that the time saved by doing things concurrently is greater than the overhead we pay to set up and coordinate that concurrency; this is the basic point formalized by Amdahl's law [2]. There are a few terms worth fixing now, since otherwise the rest of the report gets slightly muddy. A task is a unit of work we want done, a worker is something that does tasks, and a core is a hardware resource.

We want to parallelize because hardware trends have made parallelism increasingly central to performance: modern machines expose multiple cores and other parallel resources, but serial programs do not (and generally cannot) automatically make good use of them [3]. When a computation can be split into mostly independent pieces, parallelization lets us exploit the structure of modern hardware to reduce wall time. In `loo`, many of the places where we parallelize are "embarrassingly parallel"–that is, the work can be split up in a manner where tasks are independent and easy to compute concurrently [1].

It is also useful to distinguish multiprocessing from multithreading [1]. In multiprocessing, we run work in separate processes. These processes usually do not share memory by default, so objects may need to be copied or serialized before workers can use them [1]. In multithreading, multiple threads run inside the same process and share memory. This can avoid some copying, but shared memory also introduces its own problems around synchronization and thread safety [1]. In R package code, the user-facing parallelization we usually deal with is process based: for example, base R's `parallel` package can use forking on Unix-like systems or socket clusters across platforms.

It is important to note that parallelization is not free. We have to start workers, send them data, schedule work, collect results, and maybe serialize objects along the way. In the worst case, this means parallel code can be slower than serial code. This is especially plausible when tasks are small, when there are many objects to copy, or when the serial part of the computation is still large. Amdahl's law captures this basic limitation–even with infinitely many workers, the non-parallel part of the program bounds the possible speedup [2]. So the relevant question is not merely whether a piece of code can be parallelized, but whether parallelizing it actually improves runtime or memory use.

Parallelization in R

Broad overview/history of how to parallelize in R.

`mirai`

The parallelization package we are using is `mirai` [4], self described as a "Minimalist Async Evaluation Framework for R". In `mirai`, workers area called daemons. Before we briefly describe how `mirai` works at a high level, it is useful to mention some of its benefits which led us to select it as our parallel backend.

Why `mirai`?

`mirai` very naturally allows for heterogenous compute through a clean API. What this means in practice is that, once we migrate to `mirai`, users will be able to run `loo` code in parallel on their laptop, on remote computers, on a HPC system, or any zany mixture ([4], specifically see sec. 6 in the Reference Manual). Decoupling the execution profile from `loo` code, especially using the API `mirai` exposes, allows for users to more easily exploit their available computational resources without much hassle.

`mirai` is also widely used across major R packages–for example, recently landing in `purrr` ([5], [6], see `in_parallel()`). `mirai` is also "the first official alternative communications backend for base R's `parallel` package." [4], and is used in other major packages like `shiny` ([4], see the vignette), and in `tune`, which is a vital part of the `tidymodels` workflow ([7], see the vignette).

How does `mirai` work?

High level overview of how `mirai` works.

Link to appendix which goes more in depth on internals, NNG and all that

Serialization

High level: what it is, how its used, how it is related to `mori`, why we can't use for model methods. Link to appendix experiment proving we can't use for model methods.

RNG

One thing worth discussing for a moment is random number generation in `mirai`, which uses the L'Ecuyer PRNG. The details around RNG in `mirai` are pretty straightforward, as we can see through a few very simple examples.

Firstly, even if we set the same seed, we won't get the same results from `lapply()` and `mirai_map()`.

```
set.seed(0)
lapply(1:4, rexp)
```

```
[[1]]
[1] 0.1840366

[[2]]
[1] 0.1457067 0.1397953

[[3]]
[1] 0.4360686 2.8949685 1.2295621

[[4]]
[1] 0.5396828 0.9565675 0.1470460 1.3907351
```

```
set.seed(0)
mirai::daemons(4)
mirai::mirai_map(1:4, rexp)[]
```

```
[[1]]
[1] 0.8086959

[[2]]
[1] 0.6304609 0.2852141

[[3]]
[1] 0.8505149 0.2523770 0.7789137
```

```
[[4]]
[1] 1.6033540 0.7051893 0.2494851 0.6181817
```

```
mirai::daemons(0)
```

Note that by default if `daemons()` is called without setting a seed, the RNG streams aren't reproducible (just like results in serial execution without setting a seed)–ensuring "statistical validity but not numerical reproducibility between runs" [8]. If we rerun the exact same `mirai` code, we do get the same results:

```
set.seed(0)
mirai::daemons(4)
mirai::mirai_map(1:4, rexp)[]
```

```
[[1]]
[1] 0.8086959

[[2]]
[1] 0.6304609 0.2852141

[[3]]
[1] 0.8505149 0.2523770 0.7789137

[[4]]
[1] 1.6033540 0.7051893 0.2494851 0.6181817
```

```
mirai::daemons(0)
```

However, changing the number of daemons alters our results, which is obviously problematic:

```
set.seed(0)
mirai::daemons(2)
mirai::mirai_map(1:4, rexp)[]
```

```
[[1]]
[1] 0.8086959

[[2]]
[1] 0.6304609 0.2852141
```

```
[[3]]
[1] 0.6358730 1.8042829 0.7560354

[[4]]
[1] 3.73047155 0.73100788 0.04516653 0.12108167
```

```
mirai::daemons(0)
```

Setting a seed manually when launching daemons gets reproducibility (stability over runs), "regardless of the number of daemons used." [8].

```
mirai::daemons(4, seed = 0)
mirai::mirai_map(1:4, rexp)[]
```

```
[[1]]
[1] 1.024857

[[2]]
[1] 0.09307937 0.52138402

[[3]]
[1] 2.3136494 0.3730846 1.6181639

[[4]]
[1] 0.8212031 0.8856498 3.6420851 1.5720858
```

```
mirai::daemons(0)

mirai::daemons(2, seed = 0)
mirai::mirai_map(1:4, rexp)[]
```

```
[[1]]
[1] 1.024857

[[2]]
[1] 0.09307937 0.52138402

[[3]]
[1] 2.3136494 0.3730846 1.6181639

[[4]]
[1] 0.8212031 0.8856498 3.6420851 1.5720858
```

```
mirai::daemons(0)
```

In essence, reproducibility is the users' responsibility, akin to compute setups. This further stresses the fact that documentation is a major part of this project.

Serial `mirai` execution

Currently, parallelization in loo goes through three paths [9]:

1. Serial execution using `lapply()`
2. Forking through `parallel::mclapply()`
3. Fresh processes through `parallel::parLapply()`

The serial execution path is taken when the `cores` argument is equal to 1, and forking is only available on Linux and MacOS. Since `mirai` [4] neatly subsumes paths 2 and 3, a question we had was what would the performance hit be if we also removed 1–that is, if a user requested serial execution, what do we lose by using `mirai` with a single core.

```
log_ratios <- -loo::example_loglik_matrix()
S <- nrow(log_ratios)
N <- ncol(log_ratios)
tail_len <- loo:::n_pareto(r_eff = rep(1, N), S = S)

idx <- seq_len(N)

work <- function(i) {
  loo:::do_psis_i(
    log_ratios_i = log_ratios[, i],
    tail_len_i = tail_len[i]
  )$pareto_k
}

mirai::daemons(1)

results <- bench::mark(
  vapply = vapply(idx, work, numeric(1)),
  lapply = lapply(idx, work),
  mirai = mirai::mirai_map(
    idx,
    work,
    log_ratios = log_ratios,
    tail_len = tail_len
```

```
  )[],
  vapply_to_list = vapply(idx, work, numeric(1)) |> as.list(),
  purrr_map = purrr::map_dbl(idx, work),
  check = same_numeric_values,
  iterations = 200
)
```

Note that `mirai_map()` and `lapply()` results are lists. `same_numeric_values()`'s definition is elided from output but simply checks that results are identical without counting unlisting as part of the benchmark (unlike `vapply_to_list`).

```
results |> summary()
```

```
# A tibble: 5 x 6
  expression          min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>     <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 vapply           9.42ms   10.5ms      93.4    3.76MB   15.2
2 lapply           9.45ms     11ms      90.5    3.44MB   15.4
3 mirai           22.43ms   25.9ms      38.2  736.67KB    0.581
4 vapply_to_list   9.59ms   11.5ms      85.2    3.44MB   14.5
5 purrr_map        9.73ms   11.5ms      85.0     4.2MB   14.4
```

```
results |> summary(relative = TRUE)
```

```
# A tibble: 5 x 6
  expression       min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>     <dbl>  <dbl>     <dbl>     <dbl>    <dbl>
1 vapply          1      1         2.45      5.23     26.2
2 lapply          1.00   1.05      2.37      4.78     26.4
3 mirai           2.38   2.46      1         1         1
4 vapply_to_list  1.02   1.09      2.23      4.78     24.9
5 purrr_map       1.03   1.10      2.23      5.84     24.8
```

NB: the `mem_alloc` column should not be interpreted as total memory usage for `mirai`: `bench::mark()` records R heap allocations via `utils::Rprofmem()`, which is not able to track daemons' memory usage [4], [10], [11].

`mirai_map()` with a single daemon has a median runtime of ~26ms, roughly 2.5x slower than the fastest in-process approach. Of course, in absolute terms the difference is negligible, adding only about 15ms. The memory usage of all in-process approaches are relatively close to one

another. `vapply()`'s small memory overhead may probably be attributed to the extra work it does validating types and lengths and such [11].

In short, it is probably a good idea to directly test the effects of serial `mirai` execution in `loo`, but it is likely that we will find that we should keep the unique single core execution path.

Memory

Memory usage and `loo`

Parallelization may lead to running out of memory (OOM). Copying the construction from that note: if you have, say, 8GB memory but spin up 16 tasks which each use 1GB, you will run out of memory. The math here is quite clear, though it is worst case and is complicated by the profile of the program–if that 1GB is peak usage and it normally doesn't go so high, and the peaks don't overlap, then it is possible for you to squeeze past without 16 gigs free. Note, however, that this 1GB usage per parallel worker is assuming that there is no memory that can be shared. If objects are reused across workers, we could using something like `mori` [12] in R to reduce the physical memory usage by mapping some objects in workers to the same shared address spaces, thus eliding copies–unless, of course, a worker modifies the object [13]. This can be loosely seen as mildly clawing back some of the benefits of forking with respect to memory. Note however, that `mori` works on a limited set of R objects, specifically "atomic vector types, lists, and data frames" [12]. See Section for empirical proof that we cannot naively use `mori` to share compiled model methods across workers.

Since OOM is a slightly tricky thing to think about, one user friendly feature we would like to implement, somewhat orthogonal to swapping to `mirai`, is having parallel functions check how much memory they need and warn users if a run might run out of memory. We could do this pretty simply by running our functions across varying input sizes and modelling peak memory usage (probably averaged over a few runs) as a function of the parameters. Of course, if we implement this we would need to test it to see how accurate and conservative our model is.

Where is parallelization currently in use?

- `function()`
  - short description
- `function()`
  - short description

Where could we benefit from introducing parallelization?

To investigate this question we use two approaches:

1) Inspecting the code base and figuring out at which points parallelization might be helpful ("analytic search")
2) Running performance tests on the loo pipeline, investigating where the bottlenecks are, and finding out whether we can improve things with parallelization.

There are some functions in loo which could benefit from parallelization, potentially to great effect, as these functions appear to do "embarrassingly parallel" work–meaning that they do independent tasks which seem amenable to speedups by executing those tasks concurrently. These as prime targets for parallelization and somewhat easily identified, we do so by inspection, as covered in the next section.

Analytic search

For each function/place in code where you think parallelization might help, do: * Introduction * short description of function * why do you think that parallelization can improve things? * Experiment: * implement parallelization * record time (define which time you are using) for parallel and unparallel version * report results (how much speed up we can get?) * Short conclusion

We can see in `elpd_loo_approximation()` that we don't push `cores` through to `loo.function()` so right now, this function only parallelizes if users set the `mc.cores` options as `loo.function()` will pick this up when its called without specifying `cores`. This is a super simple fix and could speed up some code, depending on how users tend to set their `cores`.

Later on in the same function, we can see that parallelization had been thought of but not implemented yet for the delta method approximations in that function (`waic_grad`, `waic_grad_marginal`, `waic_hess`). Each of these loop row-wise over data and shouldn't be too difficult to parallelize. More careful thought must go into determining if parallelization is even worth it, and more careful scrutiny of the callsites of `elpd_loo_approximation()` to ensure it doesn't accidentally induce nested parallelization (see Section)–though at first blush this seems unlikely.

In `waic.function()` `loo` simply doesn't parallelize the work, relying only on a base `lapply()`. This seems like a pretty easy fix, and there doesn't seem to be any reason to not use parallelization. Again, should be careful and benchmark to ensure we aren't accidentally eating performance or RAM, but this could be a large, free win.

Lastly, in `ap_psis.array()`, which mostly just converts array arguments to matrices before passing the buck to the matrix method, we can note that cores gets hardcoded to 1 when

calling `ap_psis.matrix()`. There doesn't seem to be any real reason to do this, eating the input `cores` like in the first example; unlike there though, we pass `cores = 1`. This is not a huge issue because users probably don't use this function, since most times they would be working directly with matrixes. Further evidencing this hypothesis is this line which should error out since it references a nonexistent `r_eff`. That obviously needs to be fixed, and this function tested.

Performance testing

- short description of approach for running performance test
- run performance test
- report results
- identify bottlenecks
- provide first ideas whether any of the identified bottlenecks can benefit from parallelization

Vectorization

When browsing the codebase to find places where parallelization could be helpful (see Section), we found two potential speedups–in these places, we may be able to benefit from *vectorizing* functions instead of parallelizing a loop. Vectorization is advantageous in these spots as we would be rewriting R loops as optimized matrix operations [14, Secs. 24.5–24.7].

> Matrix algebra is a general example of vectorisation. There loops are executed by highly tuned external libraries like BLAS. If you can figure out a way to use matrix algebra to solve your problem, you'll often get a very fast solution.

Of course, for these potential improvements, we should first write tests for correctness, then write benchmarks in the PR.

The more important of the two is in `pseudobma_weights()`, when performing a Bayesian bootstrap. This function is the backend for `loo_model_weights()`, and is used to average models using psuedo-Bayesian model averaging, optionally using a Bayesian bootstrap (which is referred to as psudo-BMA+). This pseudo-BMA+ path is the one we care about as it has the elidable loop. We currently loop over Bayesian bootstrap iterations where it seems quite plausible to swap to matrix operations. This could potentially be a large speedup as the loop is large, being of length `BB_n`, which defaults to 1,000 replications. There don't appear to be any reallocations in the loop, so the speedup will probably be dominated by the efficiency of matrix ops [14, Secs. 5.3.1, 24.6]. This is tracked in #XXX.

Similarly, though less interesting, is a gradient calculation in `stacking_weights()`, just above the previous function. `stacking_weights()` is similarly used for combining models using stacking, that is, fitting a linear model to weigh the contending models. The gradient function is used for solving for those weights. However, the speedup here is less interesting because we

are looping over `K`, the number of models being compared, which is usually small. However, we may as well take the (potential) free performance. This is tracked in #XXX.

These two loops seem like low hanging fruit and well worth the time to implement.

Nested parallelization

When updating `loo` we wish to avoid "nested parallelization", that is, attempting to compute something in parallel, when already in a parallel process. Take this nonfunctioning `mirai` code as an example:

```
# parallel over xs
mirai_map(
  xs,
  # parallel over x
  function(x) mirai_map(x, mean)[]
)
```

We first are doing something parallely over values of `xs`, but then we try to work in parallel again for each value from `xs`! The main problem here is oversubscription: we spawn `n` outer workers, let say, then for each we spawn another `m` inner workers. In the package context though, this implementation would be far more complex, and would probably be hidden completely from users. This becomes a problem when a user sets `n`, the outer parallel workers, to # cores - 1[1], as the actual number of parallel tasks could be as large as $n * m$. Oversubscription is bad because, in this context, we are mostly running CPU heavy tasks where each process takes time and needs its own core. Oversubscription is promising more parallelization than what the hardware can provide–note that parallelization usually has some fixed startup costs as well (see Section), so we would probably lose performance. In `loo` specifically, we must be careful when parallelizing functions to ensure we do not accidentally have nested parallelization. Note that `mirai` exposes a function, `on_daemon()`, which may be helpful to avoid nested parallelization [4]. However, note that this only accounts for nested `mirai` runs, and not nested parallelization on the whole.

Where do we currently find nested parallelization in `loo`?

- short description of function
- do you see any challenges with the identified functions?
- should nested parallelisation run parallel or sequential?

[1]This is a common heuristic for number of threads/daemons/workers/etc. to use because it leaves 1 core free to do normal computer things, lest the system become unresponsive. Note this doesn't account for memory usage at all–if you have 8GB memory but send out n tasks, each using 2GB, you may run out of memory (see Section).

– provide experiments to stress your statement

Potential problems with packages that make use of `loo` such that nested parallelization can occur?

However, we know that other packages also call `loo` functions, which is another potential vector for nested parallelization. To this end, it seems like `brms` is the only (Stan) package where this could even be vaguely possible, but doing so seems to necessitate wilful parallelization not provided natively by `brms`. This merits further investigation, but it seems like we can (softly) table concerns regarding external nested parallelization for now.

Make a list of packages of which you have thought of: * package name * Any issues that you see? * (make a short note even if you don't see issues)

Appendix

`b3`: Bayesian Branch Benchmarking

What follows is a proposal for `b3`, a benchmarking tool designed to replace `touchstone` in Stan R packages and is far more flexible and broader in scope. `b3` is included here in the appendix as it could potentially improve our PR-level performance benchmarking, both for time and memory usage.

`b3` aims to measure language-agnostic end-to-end branch-vs-baseline performance diffs. `b3`'s philosophy is akin to touchstone [15], but shucks the R focus–`b3` handles orchestration, measurement, and reporting, while users must provide their project runtime, dependency setup, and benchmark command. `b3` compares complete repository states: if the candidate branch changes the benchmark, data, config, or dependencies, that change is part of the measured result.

Core

`b3` is one prebuilt native binary.

Users only set up their project runtime and declare their benchmarks.

R users set up R. Node users set up Node. Rust users set up Rust. Python users set up Python.

Setup and benchmark commands are opaque subprocesses specified by the user which `b3` simply executes. On Unix they may be Bash; on Windows they may be PowerShell; for R projects they may be `Rscript` directly.

`b3` handles:

git worktrees runtime preset environment variables optional setup commands paired randomized benchmark runs timing process-tree peak memory polling Bayesian bootstrap posterior summaries

Evaluation Environment

Similar to `touchstone`, `b3` separates environment isolation from dependency caching. Each variant gets its own environment inside its worktree, so baseline and candidate can resolve different dependency graphs safely. Download caches are shared across variants and CI runs, so unchanged packages do not need to be downloaded or rebuilt repeatedly. Of course, `b3` does not resolve dependencies itself–it exposes runtime presets that configure standard env/cache locations, then runs the user's setup command. Users can always opt into specifying everything manually. Regardless, dependency resolution remains the responsibility of the project's normal tooling: `rv`, `renv`, `pak`, `uv`, `npm`, `cargo`, etc.

`b3` can optionally run a cleanup command before each measured run. By default, dependency environments are reused across runs, but benchmark-created state is the user's responsibility. Benchmarks should either avoid persistent side effects which impact performance, or configure cleanup so each measured run starts from a comparable state.

Measurement

`b3` can run one or more named benchmarks. Each benchmark gets its own paired randomized baseline/candidate runs and its own posterior summaries for time and memory usage. For each paired block, randomize order: baseline then candidate, or candidate then baseline. Each command runs from its own worktree.

`b3` records:

elapsed_sec average_memory_bytes peak_memory_bytes exit_status

Runs with non-zero exit status are reported separately and excluded from posterior summaries unless explicitly configured otherwise. Note that memory measurement is best-effort and potentially platform-dependent. Short-lived spikes, detached subprocesses, permission boundaries, and OS-specific memory accounting may affect the reported value. Limitations/scope will be made abundantly clear in the API.

Analysis

For each metric, B3 computes paired log ratios:

$$d_i = log(candidate_i/baseline_i)$$

B3 uses a Bayesian bootstrap over paired blocks: each posterior draw samples Dirichlet weights over blocks, computes the weighted mean of the paired log ratios. B3 also employs mild shrinkage toward 0 (which can be disabled).

B3 reports:

median candidate/baseline ratio median percent change credible intervals P(candidate worse) P(candidate >1%, >5%, >10%, >20% worse)

B3 also reports order-stratified posterior summaries for baseline-first and candidate-first paired blocks. If the order-stratified effects disagree materially, this *may* mean that the results are unstable and that the number of runs should be increased. The benchmarks/commands may be suspect as well. In the future, B3 might expand to a more complex model which can better capture order effects.

Misc.

`b3` will also help scaffold GHA workflows to quickstart using `b3` for PR-level performance reports. Extensions to the project include a more complex model, wrappers around the core CLI in various host languages to facilitate usage, first-class support for standing up non-GHA CI/CD workflows, and so on.

`mori` and model methods

Session Info

```
sessioninfo::session_info(
  pkgs = c("loo", "mirai", "bench", "purrr", "sessioninfo"),
  info = c("platform", "packages"),
  dependencies = FALSE
)
```

```
- Session info -------------------------------------------------------------
 setting  value
 version  R version 4.6.0 (2026-04-24 ucrt)
 os       Windows 11 x64 (build 26200)
```

```
 system   x86_64, mingw32
 ui       RTerm
 language (EN)
 collate  English_United States.utf8
 ctype    English_United States.utf8
 tz       America/Los_Angeles
 date     2026-06-05
 pandoc   3.8.3 @ c:\\Program Files\\Positron\\resources\\app\\quarto\\bin\\tools/ (via rmarkd
 quarto   1.9.36 @ C:\\Users\\visru\\AppData\\Local\\Programs\\Quarto\\bin\\quarto.exe

- Packages -------------------------------------------------------------------
 package     * version    date (UTC) lib source
 bench         1.1.4      2025-01-16 [1] RSPM (R 4.6.0)
 loo           2.9.0      2025-12-23 [1] RSPM (R 4.6.0)
 mirai         2.7.0      2026-05-08 [1] RSPM (R 4.6.0)
 purrr         1.2.2      2026-04-10 [1] RSPM (R 4.6.0)
 sessioninfo   1.2.3.9000 2026-06-02 [1] Github (r-lib/sessioninfo@d4e98a4)

 [1] C:/Users/visru/AppData/Local/R/win-library/4.6
 [2] C:/Program Files/R/R-4.6.0/library

------------------------------------------------------------------------------
```

References

[1] P. Pacheco and M. Malensek, *An introduction to parallel programming*. Elsevier, 2022. doi: 10.1016/B978-0-12-804605-0.00002-6. Available: https://linkinghub.elsevier.com/retrieve/pii/B9780128046050000026. [Accessed: June 02, 2026]

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the april 18-20, 1967, spring joint computer conference*, in AFIPS '67 (spring). New York, NY, USA: Association for Computing Machinery, 1967, pp. 483–485. doi: 10.1145/1465482.1465560. Available: https://doi.org/10.1145/1465482.1465560

[3] K. Asanovic *et al.*, "A view of the parallel computing landscape," *Communications of the ACM*, vol. 52, no. 10, pp. 56–67, Oct. 2009, doi: 10.1145/1562764.1562783. Available: https://dl.acm.org/doi/10.1145/1562764.1562783. [Accessed: June 01, 2026]

[4] C. Gao, *Mirai: Minimalist async evaluation framework for r*. 2026. Available: https://mirai.r-lib.org

[5] D. V. Charlie Gao Hadley Wickham and L. Henry, "Parallel processing in purrr 1.1.0," July 10, 2025. Available: https://tidyverse.org/blog/2025/07/purrr-1-1-0-parallel/

[6] H. Wickham and L. Henry, *Purrr: Functional programming tools*. 2026. Available: https://purrr.tidyverse.org/

[7] M. Kuhn, *Tune: Tidy tuning tools*. 2026. Available: https://tune.tidymodels.org/

[8] C. Gao, "Mirai 2.5.0," Sept. 05, 2025. Available: https://tidyverse.org/blog/2025/09/mirai-2-5-0/

[9] A. Vehtari *et al.*, "Loo: Efficient leave-one-out cross-validation and WAIC for bayesian-models." 2025. Available: https://mc-stan.org/loo/

[10] J. Hester and D. Vaughan, *Bench: High precision timing of r expressions*. 2025. Available: https://bench.r-lib.org/

[11] R Core Team, *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing, 2026. Available: https://www.R-project.org/

[12] C. Gao, "Mori: Shared memory for r objects," Apr. 23, 2026. Available: https://opensource.posit.co/blog/2026-04-23_mori-0-1-0/. [Accessed: May 26, 2026]

[13] C. Gao, *Mori: Shared memory for r objects*. 2026. Available: https://shikokuchuo.net/mori/

[14] H. Wickham, *Advanced r*, 2nd ed. Chapman; Hall/CRC, 2019. doi: 10.1201/9781351201315. Available: https://adv-r.hadley.nz/

[15] L. Walthert and J. Wujciak-Jens, *Touchstone: Continuous benchmarking with statistical confidence based on 'git' branches*. 2026. Available: https://github.com/lorenzwalthert/touchstone